Logic Programming

Rodica Potolea
Camelia Lemnaru
Richard Ardelean

Lecture #7

Difference Lists

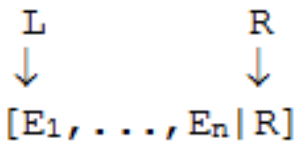
TECHNICAL UNIVERSITY OF CLUJ-NAPOCA

Agenda

- **Difference lists**
  - Quicksort
  - queues
  - Flattening lists

Difference Lists

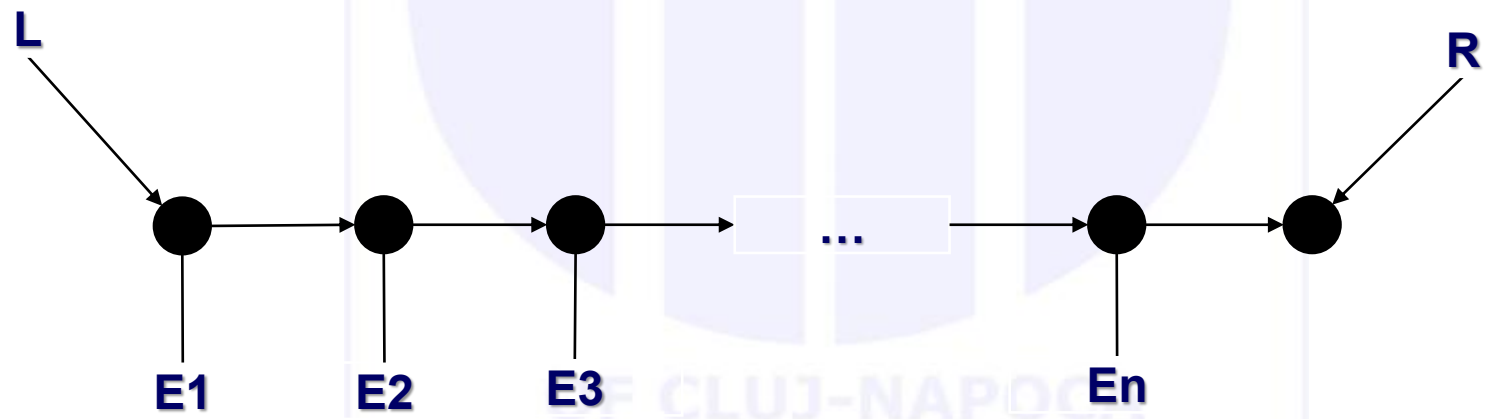
- Another type of "irregular" structures
- Allow access to both the start and the END of lists ☺
- Named as lists with **Start** and **End** element
  - (that is, besides the variable ="pointer" at the beginning of the structure, we have a variable to "point" at the end of the list).
  - Similar to imperative languages where we have lists with pointers in the beginning and end of the list
- Allow for increased performance
- Difference list:



- D, denoted as L-R (sometimes, in code, it is preferred to be represented as 2 arguments, L and R) is:
  - $D = L-R = [E_1,...,E_n]$
  - L "points" at the start, R at the "end"

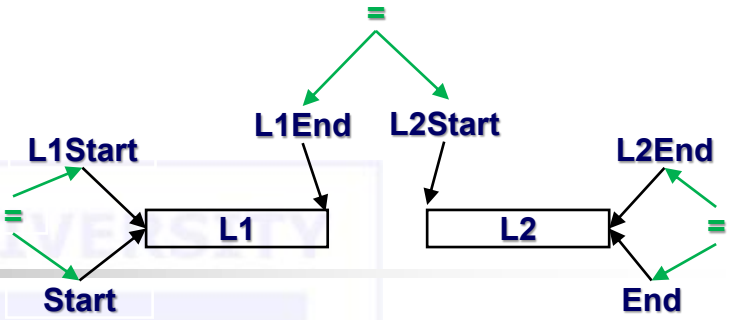
Difference Lists

L R
↓ ↓
[E1, . . . , En | R]



L = [E1, E2, …, En | R]

Append




```
% append/3 (+List1, +List2, -OutList)
append([], L2, L2).
append([H|T], L2, [H|R]):-
        append(T, L2, R).
```

For Difference Lists (DL), each list has 2 arguments.

```
% append_DL    % Number of arguments?
% /6 (+L1Start, +L1End, +L2Start, +L2End, -OutStart, -OutEnd)
append_DL(LS1, LE1, LS2, LE2, RS, RE):-
        RS = LS1,
        LE1 = LS2,
        RE = LE2.
```

Append

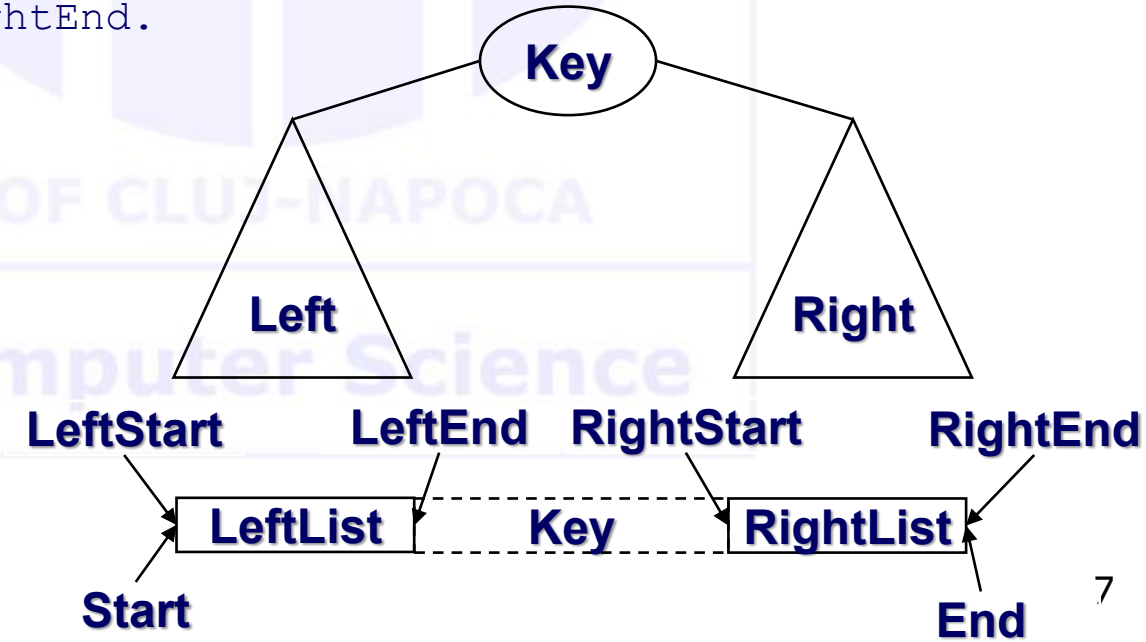
```
% append_DL/6 (+L1Start, +L1End, +L2Start, +L2End, -OutStart, -OutEnd)
append_DL(LS1, LE1, LS2, LE2, RS, RE):-
        RS = LS1,
        LE1 = LS2,
        RE = LE2.

?- append_DL([1,2|E1], E1, [3,4|E2], E2, S, E).
     % S = [1,2|E1],
     % E1 = [3,4|E2],          % → S=[1,2|[3,4|E2]=[1,2,3,4|E2]
     % RE = E2.                % → S=[1,2,3,4|E]
S=[1,2,3,4|E]
```

Tree to list (remember solutions in Lecture 5)

% inorder/3 (+Tree, -DifferenceListStart, -DifferenceListEnd)

```
inorder(nil, Start, End):- Start = End.
inorder(t(Left,Key,Right), Start, End):-
        inorder(Left, LeftStart, LeftEnd),
        inorder(Right,RightStart,RightEnd),
        Start           = LeftStart,
        LeftEnd         = [Key|RightStart],
        End             = RightEnd.
```



Tree 2 list forward – code (Lecture #5)

% inorder1/3 (+Tree, +PartialResultList, -FinalResultList)

```
inorder1(nil,L,L).
inorder1(n(Left,Key,Right), Lin, Lout):-
     inorder1(Left, Lin, LLout),
     append(LLout,[Key],LRin),
     inorder1(Right,LRin,Lout).

inorder1(Tree,Result):-
     inorder1(Tree,[],Result).
```

Tree to list (same solution – check Lecture #5)

- Needs specialized call

```
inorder(ITree,OList):-
        inorder(ITree,OList,[]).
```

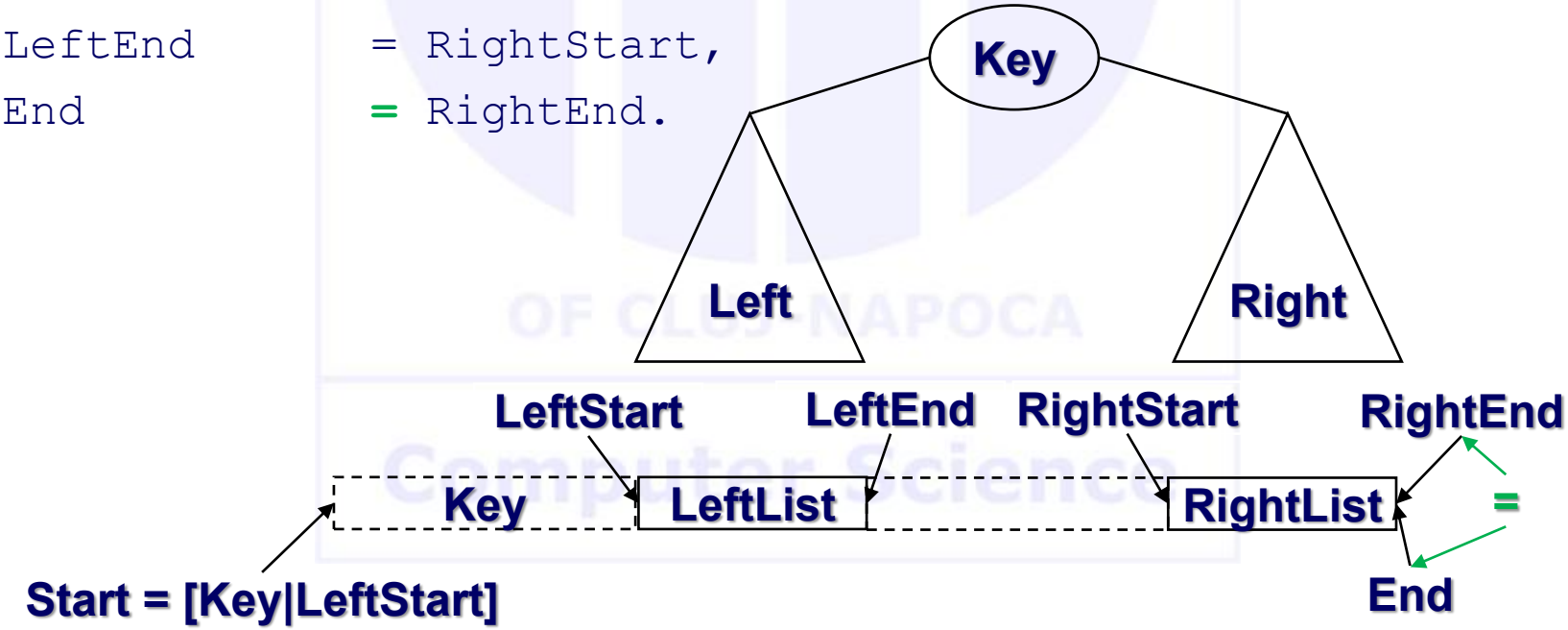
- Replace explicit unifications with implicit ones.

```
inorder(nil,L,L).
inorder(t(Left,Key,Right),LeftStart,RightEnd):-
        inorder(Left,LeftStart,[Key|Intermediate]),
        inorder(Right,Intermediate, RightEnd).
```

- Same as before, yet the utilization of the same name for variables (`LeftStart`) instead of making explicit unifications (`Start= LeftStart` and the other 2).

Pre-order

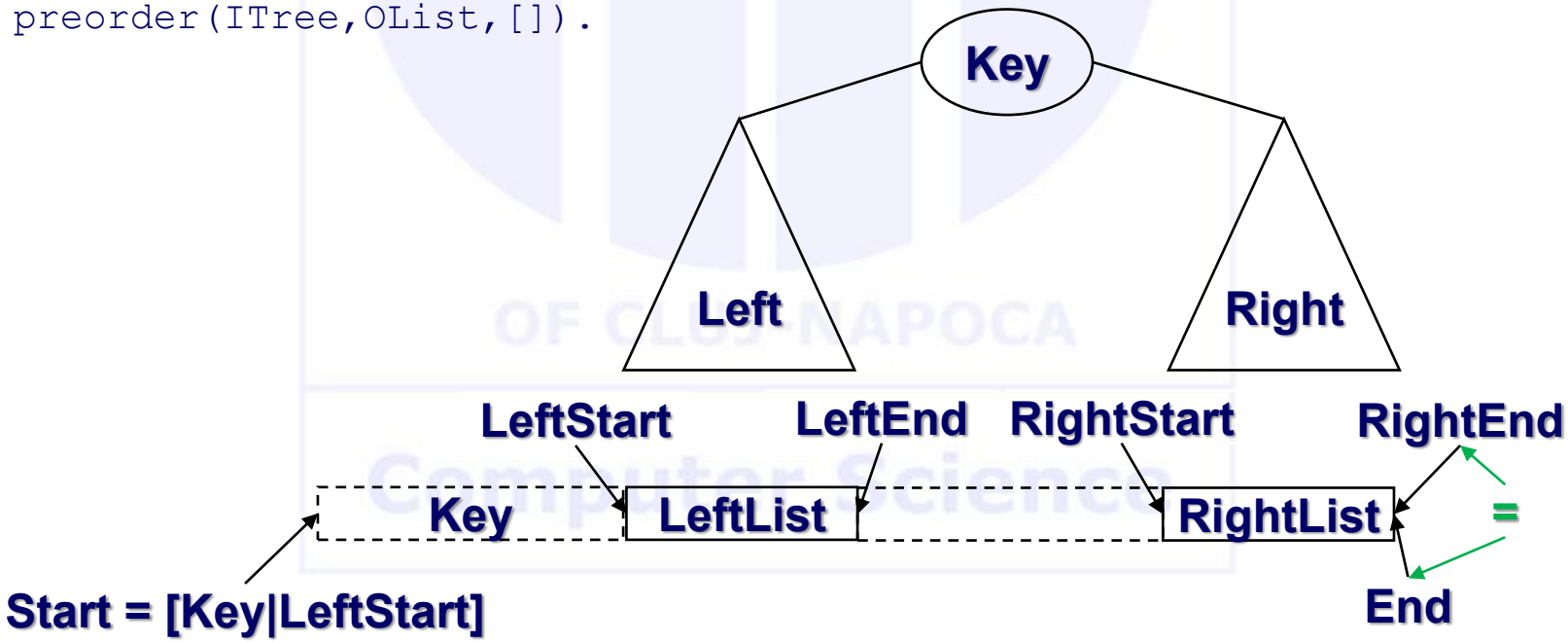
```
preorder(nil, Start,End):- Start = End.
preorder(t(Left,Key,Right), Start, End):-
        preorder(Left, LeftStart, LeftEnd),
        preorder(Right,RightStart,RightEnd),
        Start          = [Key|LeftStart],
        LeftEnd        = RightStart,
        End            = RightEnd.
```



Pre-order

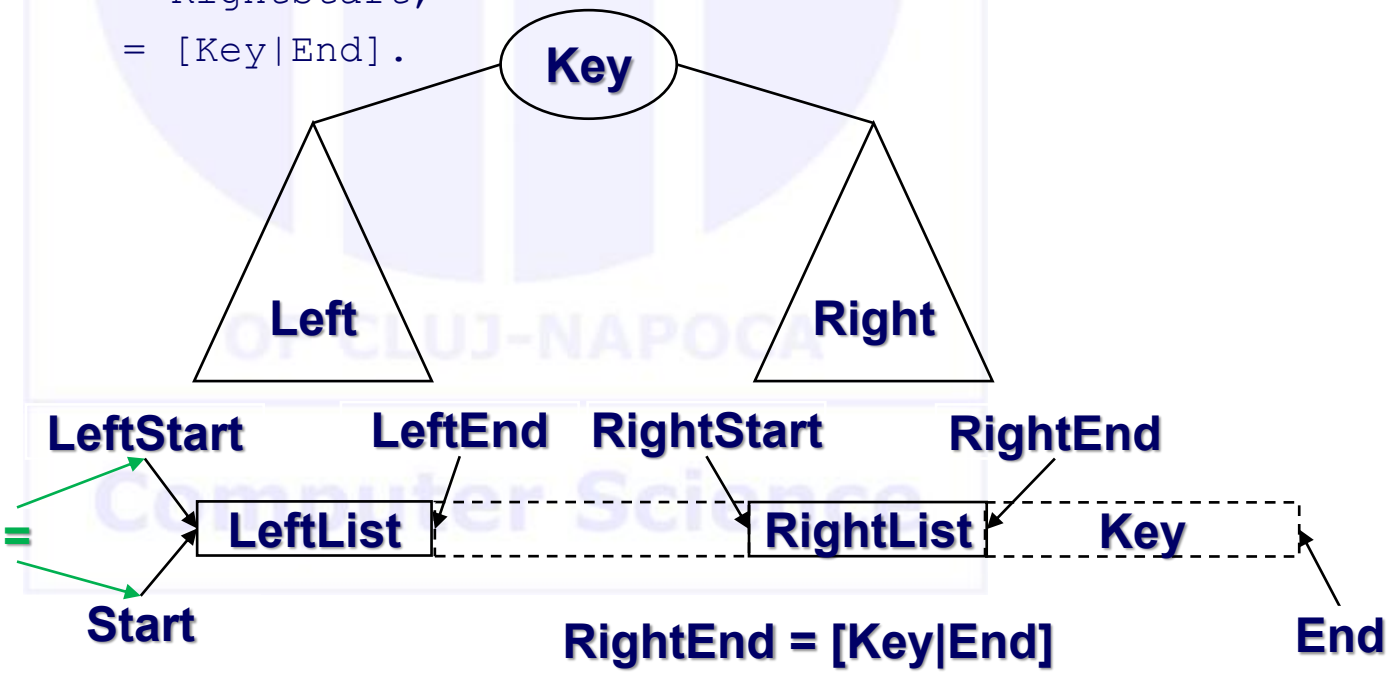
```
preorder(nil,L,L).
preorder(t(Left,Key,Right),[Key|LeftStart],RightEnd):-
        preorder(Left, LeftStart, Interm),
        preorder(Right,Interm, RightEnd).

preorder(ITree,OList):-
        preorder(ITree,OList,[]).
```



Post-order

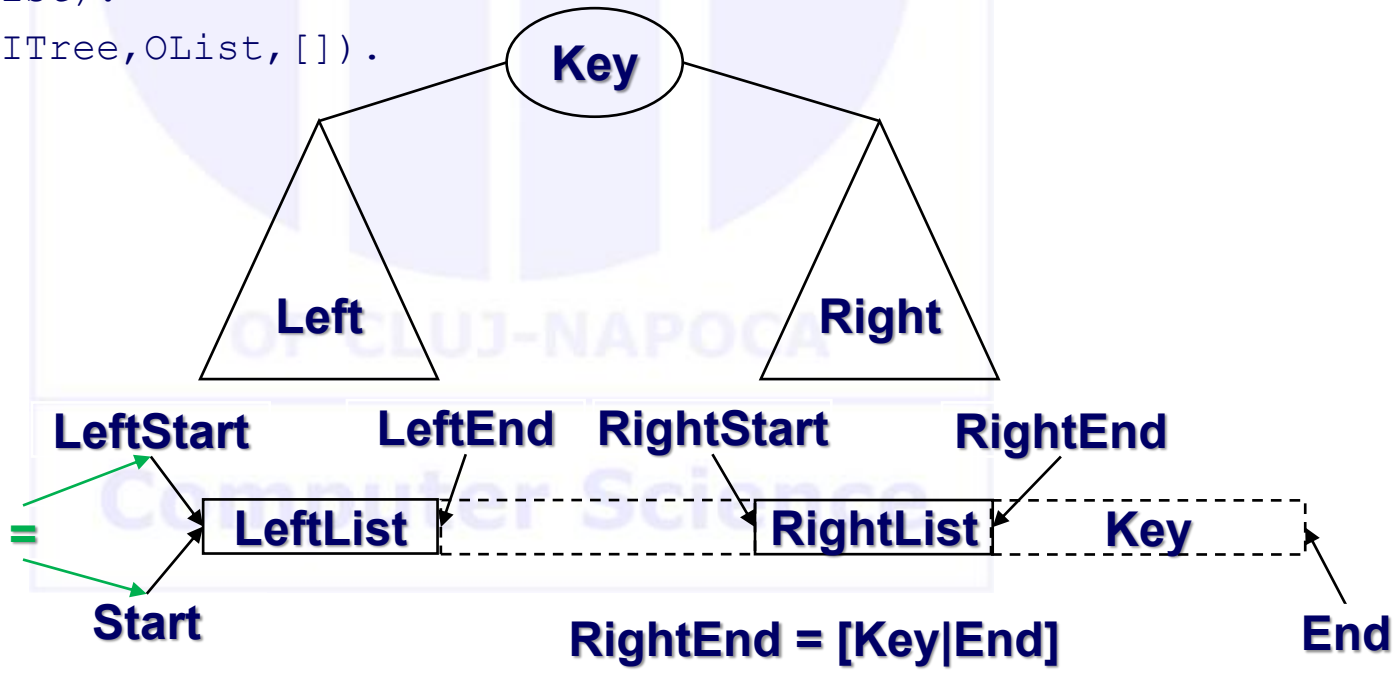
```
postorder(nil, Start,End):- Start = End.
postorder(t(Left,Key,Right), Start, End):-
        postorder(Left, LeftStart, LeftEnd),
        postorder(Right,RightStart,RightEnd),
        Start           = LeftStart,
        LeftEnd         = RightStart,
        RightEnd        = [Key|End].
```



Post-order

```
postorder(nil,L,L).
postorder(t(Left,Key,Right),LeftStart,RightEnd):-
        postorder(Left,  LeftStart, Intermediate),
        postorder(Right, Intermediate, [Key|RightEnd]).

postorder(ITree,OList):-
        postorder(ITree,OList,[]).
```



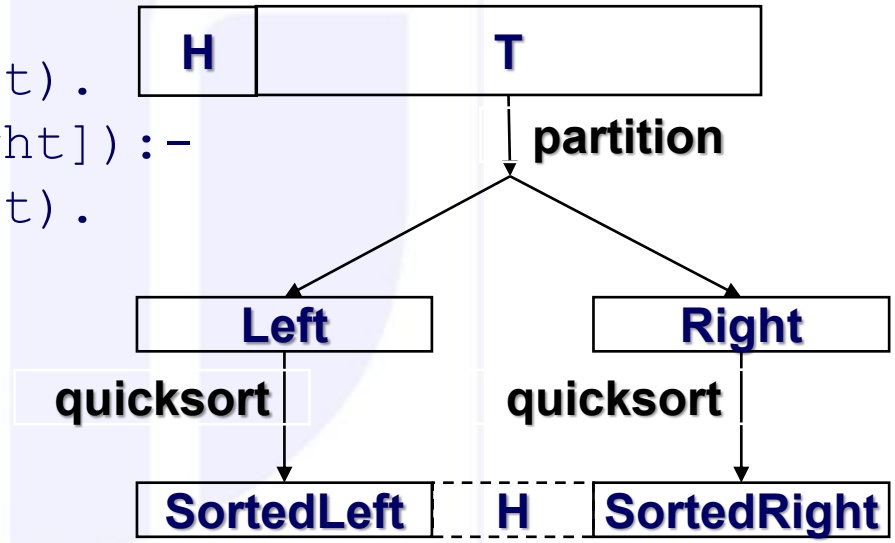
TECHNICAL UNIVERSITY OF CLUJ-NAPOCA

Difference lists

- Pair of arguments (Start,End) or Start/End, pointing to the **Start** and respectively AFTER the **End** element of the list
- Allow to speed up execution (avoid second traversal of list)
- Needs special initial call => make the warm up call by setting End on call to []
  - What if instead of [] we have some content?
- Stop condition: empty list as a difference
  - therefore, the difference empty list is between the same variable (L,L).

Quicksort

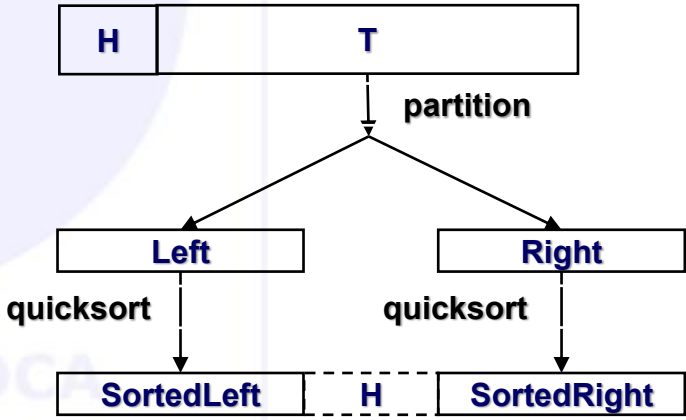
```
partition(X,[H|T],[H|Left],Right):-
      H<X,!,
      partition(X,T,Left,Right).
partition(X,[H|T],Left,[H|Right]):-
      partition(X,T,Left,Right).
partition(_,[],[],[]).
```




```
quicksort([H|T],Result):-
      partition(H,T,Left,Right),
      quicksort(Left, SortedLeft),
      quicksort(Right,SortedRight),
      append(SortedLeft,[H|SortedRight],Result).
quicksort([],[]).
```

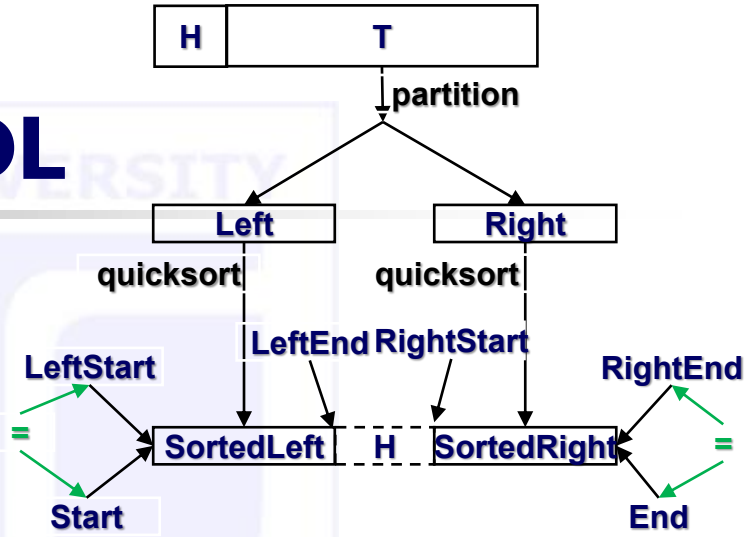
Quicksort - analysis

- A divide et impera strategy
- Cost analysis per cases:
  - O(nlgn) in best and average case (why?)
  - O(n^2) in worst (which?)
- Multiplicative constant larger than in imperative implementations!
  - Why?
  - How can this be avoided?




```
quicksort([H|T],Result):-
        partition(H,T,Left,Right),
        quicksort(Left,SortedLeft),
        quicksort(Right,SortedRight),
        % how can we skip the append
        append(SortedLeft,[H|SortedRight],Result).
quicksort([],[]).
```

Quicksort – with DL




```
% quicksort_DL1/3
% (+List, -OutListStart, -OutListEnd)

quicksort_DL1([H|T],Start,End):-
      partition(H,T,Left,Right),
      quicksort_DL1(Left, StartLeft, EndLeft),
      quicksort_DL1(Right,StartRight,EndRight),
      Start      = LeftStart,
      EndLeft    = [H|StartRight],
      RightEnd   = End.
quicksort_DL1([],L,L).
```

- Q:
  - `partition` should not return difference lists? How should we do that?
  - Oral discussion.

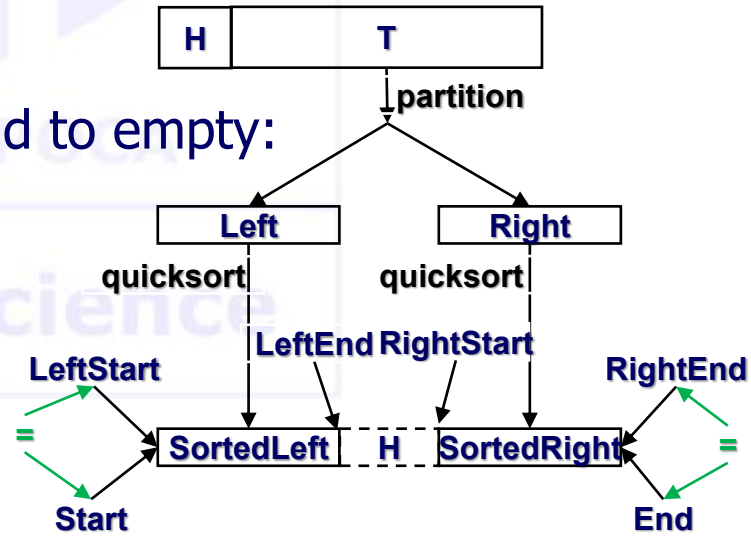
Quicksort – with DL (default unifications)

% quicksort_DL2/3 (+List, -OutListStart, -OutListEnd)

```
quicksort_DL2([H|T],Start,End):-
        partition(H,T,Left,Right),
        quicksort_DL2(Left, Start, [H|Inter]),
        quicksort_DL2(Right,Inter, End).
quicksort_DL2([],List,List).
```

- Q: needs special initial call, to initialize End to empty:

```
quicksort2(List,Result):-
        quicksort_LD2(List,Result,[]).
```



Queues with DL
(section 15.4 in The art of Prolog – Shapiro)

% enqueue/5 (+El2enQ, +QBeforeStart, +QBeforeEnd, -QAfterStart, -QAfterEnd)

```
enqueue(El,Start,[El|End],Start,End).
```

Means:

```
enqueue(El,QS,QE,Start,End):-
        QS=Start,     % after adding in the Q, it starts in the same place
        QE=[H|End],   % before adding, the list ends IN FRONT
        H=El.         % of the item to be added
```

How to query it?

```
?- enqueue(Item,QS,QE,Start,End).
```

In the usual employment `QS` and `QE` are the Start/End element in the Queue before the call (known arguments), and the `Start`/`End` element in the Queue after the call (unknown arguments at call time).

Queues with DL
(section 15.4 in The art of Prolog – Shapiro)

```
% dequeue/5 (-El2deQ, +QBeforeStart, +QBeforeEnd, -QAfterStart, -QAfterEnd)

dequeue(El,[El|Start],End,Start,End).
```

Means:

```
dequeue(El,QS,QE,Start,End):-
        QS=[H|Int],   % extracted element H from the front of Q
        El=H,         % assign it to our argument
        Int=Start,    % Q without Start element is our StartQueueAfter
        QE=End.       % list ends are the same as removal is from front
```

How to query it?

```
?- dequeue(Item,QS,QE,Start,End).
```

- In the default meaning, `Item` gets instantiated at the end of the execution.
- What happens if we ask to dequeue some specific element (say 7), yet, it is NOT the Start in the queue? That is, we specify `Item`.

Types of lists

- Regular/Complete lists - ended in []
- Incomplete lists - ended in variable
- Difference lists
- Write specific predicates to make transformations among each of the 3 known types of lists (section 7.3 of the textbook):
  - **Complete to incomplete**
  - Complete to difference
  - Incomplete to complete
  - **Incomplete to difference**
  - Difference to complete
  - Difference to incomplete

TECHNICAL UNIVERSITY OF CLUJ-NAPOCA

Deep lists

- What is a deep list?

```
L=[1,1,[2,2,2,[3,3,[4],3],2,2,[3,[4,[5,5,5]]],2]]
```

- Levels? How do we know?

- Flatten deep lists. Meaning?

```
L=[1,1,2,2,2,3,3,4,3,2,2,3,4,5,5,5,2]
```

- How, without knowing structure / number of levels?

Flatten Deep lists

```
% deep2flat/2 (+DeepList, -FlatList).

deep2flat([],[]):-!.
deep2flat([H|T],[H| Tflat]):-
      atomic(H),!,
      deep2flat(T, Tflat).
deep2flat([H|T],R):-
      deep2flat(H,Hflat),      % H is a list
      deep2flat(T,Tflat),
      append(Hflat, Tflat, R).
```

```
[q] ?- deep2flat([1,1,[2,2,2,[3,3,[4],3],2,2,[3,[4,[5,5,5]]],2]], R)
R=              [1,1, 2,2,2, 3,3, 4, 3, 2,2, 3, 4, 5,5,5,   2]
```

Flatten Deep lists – with difference lists - explicit

```
% deep2flat_dl/3 (+DeepList, -FlatListStart, -FlatListEnd).

deep2flat_dl([],    Start, End):- !, Start = End.
deep2flat_dl([H|T],Start, End):-
      atomic(H),!,
      deep2flat_dl(T, TStart, TEnd),
      Start = [H|TStart],
      End   = TEnd.
deep2flat_dl([H|T],Start, End):-
      deep2flat_dl(H,HStart,HEnd),
      deep2flat_dl(T,TStart,TEnd),
      Start = HStart,
      HEnd  = TStart,
      End   = TEnd.
```

Flatten Deep lists – with difference lists - default

```
% deep2flat_dl/3 (+DeepList, -FlatListStart, -FlatListEnd).

deep2flat_dl([], End, End):-!.
deep2flat_dl([H|T],[H|Tflat],End):-
            atomic(H),!,
            deep2flat_dl(T,Tflat,End).
deep2flat_dl([H|T],Start,End):-
            deep2flat_dl(H,Start,Int),
            deep2flat_dl(T,Int,End).

flatten(Deep,Flat):-
       deep2flat_dl(Deep,Flat,[]).
```

Flatten Deep lists – homework

- Try to flatten a deep least taking just elements with various properties:
  - Heads of lists ([1,2,3,4,3,4,5])
  - Atomic elements on End positions of lists ([4,3,5,2])
  - All atomic elements from lists at **odd**/**even** levels ([2,2,2,4,2,2,4,2])
  - Heads of lists at **odd**/**even** levels ([1,3,3,5])
  - Atomic elements on **odd**/**even** positions on any list
  - ….imagine ☺
  - (section 8.3 of the textbook)

```
L=[1,1,[2,2,2,[3,3,[4],3],2,2,[3,[4,[5,5,5]]],2]]
```